DATA STRUCTURES(ETCCDS202)

UNIT-1 ASSIGNMENT:FOUNDATIONS AND ALGORITHMIC ANALYSIS

-:Course Details:-

Name of school: School of Engineering and technology

Program: B.Tech(CSE AI/ML)

Course title: Data Structures

Course code: ETCCDS202

Unit title: Foundations and Algorithmic Analysis

Student name: Deepak **Roll No:**2501730097

Section: B.Tech(CSE AI/ML)-Sec-A **Submission date:** March 04,2026

Part A: Stack ADT (ADT + Operations)

- What is ADT?
  - An Abstract Data Type (ADT) is a fundamental concept in computer science that essentially acts as a "contract" for how a collection of data should behave, without dictating how that behavior is actually implemented.
- Why stack operations are usually O(1) (constant time)?
  - Stack operations like push and pop are usually O(1) because we dont care about the size of the data—we only care about the top of the stack. In a stack, we dont search through the collection or shift elements around. We are always interacting with a single, known memory location.

Part B: Factorial & Fibonacci (Recursion + Complexity)

Factorial (n!)

For a simple iterative or recursive approach, the complexity is:

- **Time: $\Theta(n)$ because it requires n multiplications.**
- **Space: $\Theta(n)$ for iterative; $\Theta(n)$ for recursive due to the call stack.**

Big- $\Omega(n)$ and Big-O(n) match here, making it tightly bound by

$\Theta(n)$.

Fibonacci (F_n)

The complexity for the naive recursive approach is:

- **Time: $O(2^n)$**
- **Space: $\Theta(n)$ due to the maximum depth of the recursion tree.**

Iterative versions or dynamic programming reduce time to $\Theta(n)$ and space to $\Theta(1)$.

Why Naive Fibonacci is Slow?

The naive recursion performs redundant calculations. To find F_n, it recalculates the same subproblems (like F_{n-2}) thousands of times across different branches. This results in an exponential growth of function calls, making it computationally expensive and inefficient for even modest values of n.

Part C: Tower of Hanoi (Recursion + Trace)

Why the number of moves is $2^n – 1$?

The recurrence relation for n disks is $T(n) = 2T(n-1) + 1$. To move n disks, you must move the top n-1 disks twice (once to the auxiliary and once to the destination) plus
one move for the largest disk. Solving this geometric series results in exactly $2^n - 1$ moves.

Time Complexity: $O(2^n)$

The algorithm generates a binary tree of recursive calls. Each disk added doubles the number of operations required, as every call for n disks triggers two subsequent calls for n-1 disks. This exponential growth means the execution time increases significantly with each additional disk, following an $O(2^n)$ pattern.

Space Complexity: O(n)

The space complexity is determined by the maximum depth of the recursion stack. Since the algorithm processes one branch at a time and the recursion depth corresponds directly to the number of disks n, the system only needs to store n stack frames at any given moment, resulting in O(n) space.

Part D: Recursive Binary Search (Divide & Conquer + Recurrence)

Time Complexity and Recurrence

The recurrence $T(n) = T(n/2) + O(1)$ represents dividing the search space in half each step and performing a constant-time comparison. By repeatedly halving the input size, the number of steps required to reach a single element is $\log_2 (n)$, leading to a complexity of O(log n).

Best, Average, and Worst Cases

- **Best Case: Occurs when the target key is exactly at the middle index of the initial array, requiring only one comparison: O(1).**
- **Average Case: Occurs when the key is found after several divisions or is absent, typically involving log(n) comparisons: O(log n).**
- **Worst Case: Occurs when the key is at the far ends of the array or not present at all, requiring the maximum number of divisions: O(log n).**

Part E: Short Paradigm Reflection

Binary Search as Divide & Conquer

Binary Search is a Divide & Conquer algorithm because it recursively breaks the search space into smaller halves. It divides the array by finding the midpoint, conquers by comparing the key to that midpoint, and reduces the problem to a single sub-array, ignoring the irrelevant half.

Fibonacci and Memoization

Memoization transforms the recursive Fibonacci calculation into Dynamic Programming by storing the results of expensive function calls. It solves the overlapping subproblems issue, preventing the algorithm from redundantly recalculating the same Fibonacci numbers multiple times, which reduces time complexity from exponential to linear.

Tower of Hanoi Recursion

The base case occurs when n = 1, where the disk is moved directly to the destination. The smaller subproblem involves moving n-1 disks to an auxiliary peg to clear a path for the largest disk, then moving those n-1 disks from the auxiliary to the final destination.

Real-Life Examples

- **Divide & Conquer: A dictionary search. You open to the middle, determine if your word is before or after that page, and repeat the process on the remaining half.**
- **Dynamic Programming: Google Maps navigation. It breaks the trip into segments and reuses previously calculated shortest paths for common intersections to find the overall fastest route.**
- **Greedy: A cashier giving change. They always choose the largest denomination possible first (e.g., a $20 bill) to minimize the total number of bills/coins handed back.**